

Математические основы алгоритмов, осень 2020 г.
Лекция 2: «Быстрая сортировка», ожидаемое время её
работы. Структуры данных: куча. Сортировка кучей.
Оценка числа сравнений при сортировке. Сортировка
подсчётом. Поразрядная сортировка. Нахождение i -го по
величине элемента массива. Метод динамического
программирования: задача о разделении стержня*

Александр Охотин

6 сентября 2021 г.

Содержание

1	«Быстрая сортировка»	1
1.1	Ожидаемое время работы при случайном выборе опорного элемента	3
2	Сортировка кучей	4
2.1	Структуры данных: куча	4
2.2	Сортировка кучей	5
3	Скорость сортировки	5
3.1	Оценка числа сравнений при сортировке	5
3.2	Сортировка подсчётом	6
3.3	Поразрядная сортировка	7
4	Нахождение i-го по величине элемента массива	8
5	Метод динамического программирования	11
5.1	Задача о разделении стержня	11

1 «Быстрая сортировка»

«Быстрая сортировка» (Quicksort), Хоар [1961] (Hoare): тоже метод «разделяй и властвуй», но размер подзадач, на которые делится задача, заранее не известен.

В сортируемом массиве выбирается *опорный элемент* (pivot element, т.е., центр вращения), и производится *разделение массива* по этому элементу — то есть, все элементы обмениваются так, чтобы вначале шли элементы, меньшие или равные опорному (в любом

*Краткое содержание лекций, прочитанных студентам 1-го курса СПбГУ, обучающимся по программам «Математика» и «МАиАД», в осеннем семестре 2020–2021 учебного года. Страница курса: http://users.math-cs.spbu.ru/~okhotin/teaching/algorithms_2020/.



Рис. 1: Джон фон Нейман (1903–1957), Энтони Хоар (род. 1934).

порядке), потом опорный, а потом — все элементы, большие, чем опорный. Это делается за линейное время.

Далее, каждая половина рекурсивно сортируется.

Позже Хоар [1981] писал: *«Необходимо отдать должное гению разработчиков Алгола 60, которые включили в свой язык рекурсию, тем самым дав мне возможность столь изящно представить миру моё изобретение».*

Алгоритм 1 «Быстрая сортировка»

Процедура `quicksort( $\ell, m$ )`

```
1: if  $m - \ell \geq 2$  then
2:    $i = \text{partition}(\ell, m)$ 
3:   quicksort( $\ell, i$ )
4:   quicksort( $i + 1, m$ )
```

Процедура `partition( $\ell, m$ )`

```
1: случайно выбрать  $p \in \{\ell, \dots, m - 1\}$  /* можно выбрать как угодно */
2:  $y = x_p$ 
3:  $x_p \leftrightarrow x_{m-1}$  /* опорный элемент сперва помещается в конец */
4:  $i = \ell$ 
5:  $j = m - 1$ 
6: while  $i < j$  do
7:   if  $x_i < y$  then
8:      $i = i + 1$ 
9:   else if  $x_j \geq y$  then
10:     $j = j - 1$ 
11:   else
12:     $x_i \leftrightarrow x_j$ 
13:  $x_i \leftrightarrow x_{m-1}$  /* переместить опорный элемент в найденную середину */
14: return  $i$  /* опорный элемент теперь находится здесь */
```

Опорный элемент можно выбрать произвольно, в том числе случайно.

Неустойчивая сортировка (не сохраняет порядок одинаковых элементов).

При удачном выборе опорного элемента массив делится примерно пополам. Если выбор почти всегда будет сколько-нибудь удачным, то время работы будет порядка $n \log n$.

При неудачном выборе в одной половине окажутся все элементы, а в другой — ни одного. Если выбор всё время будет неудачным или близким к неудачному, то порядок времени

работы будет n^2 .

1.1 Ожидаемое время работы при случайном выборе опорного элемента

Если опорный элемент выбирается случайно, то полученный алгоритм относится к классу *вероятностных алгоритмов*. *Вычисление алгоритма* — это последовательность шагов, которая состоит при каком-то определённом выборе случайных чисел. Для каждого входного массива алгоритм может проделать одно из нескольких различных вычислений: уже после первого выбора опорного элемента вычисления пойдут разными путями, в зависимости от того, какой элемент будет выбран. Некоторые вычисления, в которых опорный элемент будет выбираться неудачно, будут иметь длину порядка n^2 , а некоторые справятся за время порядка $n \log n$. Оказывается, что для вероятностного алгоритма быстрой сортировки ожидаемое время работы будет всего лишь порядка $n \log n$.

Пусть $(x_1, \dots, x_n)$ — входной массив, пусть π — одно из возможных вычислений алгоритма на нём, пусть $|\pi|$ — количество шагов в этом вычислении, и пусть $P(\pi)$ — вероятность этого вычисления. Тогда *математическое ожидание времени работы* алгоритма задаётся следующим выражением.

$$\sum_{\pi \text{ — вычисление на } (x_1, \dots, x_n)} P(\pi) \cdot |\pi|$$

Теорема 1. *Если все элементы массива различны и опорный элемент всякий раз выбирается случайно, то для всякого набора входных данных математическое ожидание времени работы составит $\Theta(n \log n)$.*

Доказательство. Время работы пропорционально числу сравнений между элементами.

Каждое производимое алгоритмом сравнение — это сравнение элемента, выбранного в качестве опорного, с одним из элементов текущего куска. Пусть y_i и y_j — два элемента уже отсортированного массива. Сколько раз они могут сравниваться за время работы алгоритма?

На нулевом уровне рекурсии, когда сортируется весь массив, и y_i и y_j , и все элементы $y_{i+1}, \dots, y_{j-1}$ между ними, находятся в одном и том же куске. Если теперь в качестве опорного элемента выбирается элемент, меньший, чем y_i , или больший, чем y_j , то на следующем уровне рекурсии все элементы $\{y_i, y_{i+1}, \dots, y_j\}$ остаются в одном куске. В какой-то момент один из этих элементов выбирается в качестве опорного. Если это будет один из промежуточных элементов y_k , где $i < k < j$, то по итогам разделения массива вокруг y_k элемент y_i попадёт в левую половину, y_j в правую, и потому сравнение между y_i и y_j никогда не будет выполнено. Если же за опорный элемент будет взят y_i или y_j , то алгоритм сравнит их один раз. Поскольку выбор одного из элементов $y_i, y_{i+1}, \dots, y_j$ по предположению равновероятен, это будет y_i или y_j с вероятностью $\frac{2}{j-i+1}$, и потому среднее число сравнений между этими двумя элементами — это $\frac{2}{j-i+1}$.

Тогда среднее число сравнений вообще — это сумма $\sum_{1 \leq i < j \leq n} \frac{2}{j-i+1}$. Она оценивается так.

$$\sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1} = \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1} = O(n \log n)$$

(на первом шаге используется замена переменных $k = j - i$, на втором — оценка $\sum_{k=1}^n \frac{1}{k} \approx \int_1^n \frac{1}{x} dx = \ln n$) $\square$

2 Сортировка кучей

2.1 Структуры данных: куча

Куча (Джон Вильямс [1964], 1930–2012): двоичное дерево, представленное в виде массива. В этом дереве различаются левый и правый потомок каждой вершины, и потому вершины на каждом уровне упорядочены. Дерево почти сбалансировано: на каждом i -м уровне, кроме, может быть, последнего, есть все 2^i вершин, а на последнем уровне есть

Вершины кучи, содержащей n элементов, пронумерованы от 1 до n . Номер корня — 1. Предок вершины i — вершина $\lfloor \frac{i}{2} \rfloor$, левый потомок — вершина $2i$, правый потомок — вершина $2i + 1$. Благодаря такой нумерации кучу удобно хранить в массиве, проиндексированном номером вершины: тогда перемещение по дереву реализуется умножением и делением на два номера вершины.

Пусть x_i — значение в вершине i . Единственное условие, налагаемое на кучу — значение во всякой вершине должно быть не больше, чем в обоих её потомках: $x_i \leq x_{2i}$ и $x_i \leq x_{2i+1}$ — это называется «кучей по минимуму» (min-heap). «Куча по максимуму» определяется аналогично.

Всякий массив можно преобразовать в кучу следующим алгоритмом (алгоритм 2). Сперва массив рассматривается в качестве кучи, в которой условие кучи нарушено в каких-то вершинах. Затем эта неправильная куча *исправляется*: индукцией по высоте поддерева последовательно обеспечивается, что каждое поддерево — правильная куча.

Каждый лист — уже сам по себе куча, ничего исправлять не надо. Далее, если у какой-то внутренней вершины i оба потомка — кучи, то вызывается процедура `heapify()` с аргументом i , делающая кучу из всего этого поддерева: элемент i «утопляется» до своего законного места.

Алгоритм 2 Преобразование массива в кучу.

Процедура `heapify(i)`

- 1: **while** значение в вершине i превосходит значение хотя бы в одном из её потомков **do**
- 2: пусть j — наименьший потомок i
- 3: обменять x_i и x_j
- 4: $i = j$ /* перейти к рассмотрению вершины j */

Процедура `make_heap()`

- 1: **for** $i = \frac{n}{2}$ to 1 **do**
 - 2: `heapify(i)`
-

Оценка времени работы процедуры `make_heap()`. Грубая верхняя оценка: процедура `heapify()` вызывается $O(n)$ раз, каждый раз она работает за время $O(\log n)$, и потому $O(n \log n)$. Но на самом деле большинство вызовов `heapify()` относятся к вершинам небольшой высоты, и потому `heapify()` для них будет работать быстрее. Более точный анализ: вершин высоты h будет $2^{\log_2 n - h}$, и для каждой такой вершины `heapify()` работает не более чем за h шагов. Отсюда получается следующая верхняя оценка.

$$\sum_{h=0}^{\log_2 n} h \cdot 2^{\log_2 n - h} = n \cdot \sum_{h=0}^{\log_2 n} \frac{h}{2^h} \leq n \cdot \sum_{h=0}^{\infty} \frac{h}{2^h} = 2n$$

2.2 Сортировка кучей

Сортировка кучей: сперва построить кучу, а потом на каждом шаге удалять из неё x_1 , заменяя его на x_n и вызывая `heapify(1)`.

3 Скорость сортировки

3.1 Оценка числа сравнений при сортировке

Сортировка, основанная на сравнении. Изначально даны элементы $x_1, \dots, x_n$. Пусть их можно только попарно сравнивать друг с другом. Каждое сравнение — это сравнение некоторых x_i и x_j («верно ли, что x_i меньше, чем x_j ?»). И работа алгоритма зависит только от результатов этих сравнений.

Алгоритм не видит самих чисел, он видит только результаты сравнений, и в каждый момент времени уже известные результаты сравнений составляют некоторые сведения о том, в каком порядке записаны эти числа. Их можно представлять в виде *множества перестановок* чисел от 1 до n — то есть, $P \subseteq S_n$.

Сперва не известно ничего, и возможна любая перестановка. После первого сравнения x_i с x_j , в зависимости от его результата, остаются или перестановки, где $p(i) < p(j)$, или перестановки, где $p(i) > p(j)$. В первом случае работает одна ветвь алгоритма, во втором — другая.

Ветвление образует дерево, где в корне — множество всех перестановок, а в потомках каждой вершины множество разбивается на два непересекающихся подмножества. В листьях же должны быть одноэлементные множества перестановок, иначе алгоритм что-то с чем-то недосравнил и выдать правильно отсортированный массив не сможет.

Такое дерево можно построить для всякого алгоритма и для каждого значения n .

Пример 1. Рассматривается сортировка вставкой.

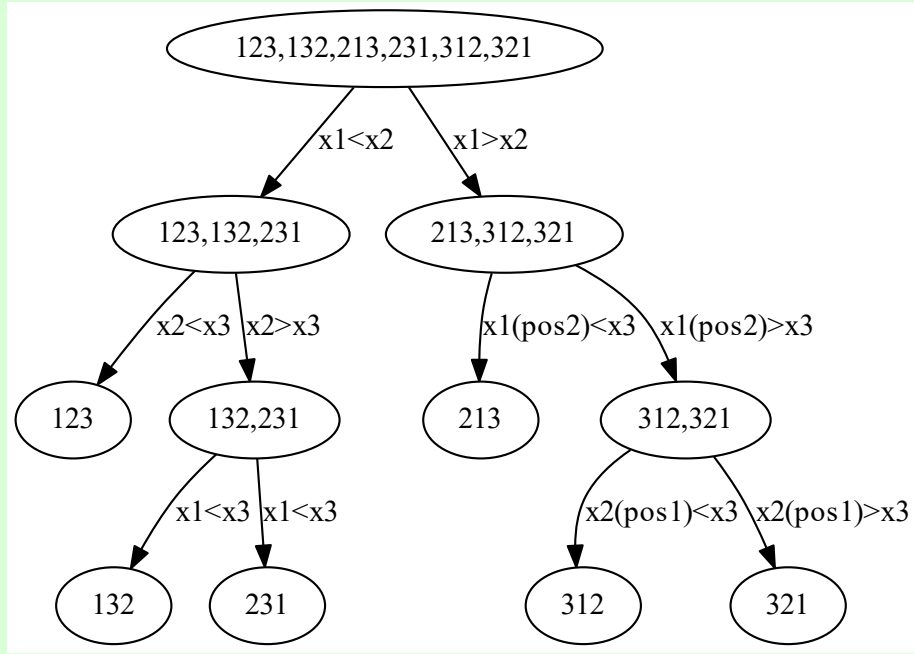
```
1: for  $j = 2$  to  $n$  do
2:    $t = x_j$ 
3:    $i = j - 1$ 
4:   while  $i \geq 1$  и  $x_i > t$  do
5:      $x_{i+1} = x_i$ 
6:      $i = i - 1$ 
7:    $x_{i+1} = t$ 
```

Пусть $n = 3$, есть 6 перестановок: $S_3 = \{123, 132, 213, 231, 312, 321\}$. Алгоритм сперва сравнивает первые два элемента (для $j = 2$ и $i = 1$). Если первый меньше второго — это множество перестановок $\{123, 132, 231\}$ — то выполняется одна ветвь вычисления, где ничего не меняется. Если же первый больше второго — множество перестановок $\{213, 312, 321\}$ — то другая, первые два элемента переставляются.

В первой ветви вычисления затем сравниваются второй и третий элементы, и вычисление делится на две ветви, $\{123\}$ и $\{132, 231\}$. В первом случае место для x_3 найдено: ничего менять не надо, и это конец работы алгоритма. Во втором случае продолжается поиск места для третьего элемента, он сравнивается с первым, и остаются случаи $\{132\}$ и $\{231\}$.

Вторая ветвь вычисления работает с массивом (x_2, x_1, x_3) , в котором уже переставлены первые два элемента. Чтобы найти надлежащее место для x_3 , он сравнивается с элементом во 2-й позиции, то есть с x_1 . По итогам сравнения множество перестановок

$\{213, 312, 321\}$ разбивается на $\{213\}$ (если $x_1 < x_3$) и на $\{312, 321\}$, причём в последнем случае второй элемент дополнительно сравнивается с третьим, и остаются случаи $\{312\}$ и $\{321\}$.



Всего перестановок $n!$, а поскольку каждое сравнение выдаёт ответ «да» или «нет», дерево двоичное. Поэтому высота дерева — не менее чем $\log_2 n!$. Используя самую приближительную оценку факториала¹ $(\frac{n}{e})^n < n! < n^n$, логарифм оценивается как $n \log_2 n - O(n)$.

Теорема 2. *Всякий алгоритм сортировки, основанный на сравнении, требует $\Omega(n \log n)$ операций сравнения.*

Можно ли работать быстрее, если не только сравнивать элементы, а делать с ними что-нибудь ещё? Зависит от типа элементов. Например, если в массиве лежат только нули и единицы, то можно за один проход посчитать количество нулей и единиц, а потом вторым проходом записать новое содержимое массива. На этой мысли основан следующий алгоритм сортировки.

3.2 Сортировка подсчётом

Простейший алгоритм, основанный не на сравнении элементов.

Пусть всякий элемент массива $x_1, \dots, x_n$ принадлежит множеству $\{1, \dots, k\}$, для достаточно малого k — точнее говоря, пусть те значения, по которым элементы должны быть

¹Верхняя оценка: $1 \cdot 2 \cdot \dots \cdot n \leq n \cdot n \cdot \dots \cdot n$. Нижняя — по индукции. Базис: $(1/e)^1 < 1!$, верно. Переход: $(n+1)! = (n+1)n! > (n+1)(\frac{n}{e})^n$ (по предположению индукции), и надо показать, что это выражение больше, чем $(\frac{n+1}{e})^{n+1}$. Действительно, $(n+1)(\frac{n}{e})^n > (\frac{n+1}{e})^{n+1}$ тогда и только тогда, когда $en^n > (n+1)^n$, что в свою очередь равносильно $(1 + \frac{1}{n}) < e$ — а это определение числа e .

отсортированы («ключи»), принадлежат этому множеству. Например, дан массив строк — имён, и надо упорядочить их по длине. В массиве может быть несколько имён одинаковой длины — и алгоритм их не различает, он видит только длины.

Алгоритм начинает с подсчёта числа раз, сколько каждое возможное значение $m \in \{1, \dots, k\}$ встречается в массиве — пусть это число y_m .

Далее, по y_m вычисляется z_m — число элементов со значением m и менее. Тогда всякий элемент исходного массива $x_i = m$ должен был бы находиться в одной из позиций $z_{m-1} + 1, \dots, z_m$. Точнее, если $x_{i_1}, \dots, x_{i_\ell}$, где $i_1 < \dots < i_\ell$, — это все элементы со значением m , то x_{i_ℓ} должен попасть в позицию z_m , а всякий последующий $x_{i_{\ell-j}}$ — в позицию $z_m - j$.

Чтобы они так и попали, алгоритм читает массив от конца к началу и при виде всякого очередного элемента x_i со значением m размещает его в новый массив по адресу z_m . Одновременно значение z_m уменьшается на единицу, чтобы следующий элемент с тем же значением попал, куда следует.

1: **for** $i = n$ to 1 **do**

2: $x'_{z_{x_i}} = x_i$

3: $z_{x_i} = z_{x_i} - 1$

Время работы: $O(n + k)$. Сортировка подсчётом устойчива.

Пример 2.

		1	2	3	4	5	6	7
x		5	4	4	5	9	4	5
		Мария	Иван	Анна	Павел	Екатерина	Олег	Ольга

	3	4	5	6	7	8	9
y	0	3	3	0	0	0	1
z	0	3	6	6	6	6	7

Состояние массивов z и x' на каждой итерации алгоритма.

x'							z		
1	2	3	4	5	6	7	4	5	9
							3	6	7
					Ольга		3	5	7
		Олег			Ольга		2	5	7
		Олег			Ольга	Екатерина	2	5	6
		Олег		Павел	Ольга	Екатерина	2	4	6
	Анна	Олег		Павел	Ольга	Екатерина	1	4	6
Иван	Анна	Олег		Павел	Ольга	Екатерина	0	4	6
Иван	Анна	Олег	Мария	Павел	Ольга	Екатерина	0	3	6

3.3 Поразрядная сортировка

Поразрядная сортировка (radix sort), тоже без сравнений.

Пусть элементы сравниваются по строкам длины ℓ над алфавитом Σ из k символов.

Тогда делается ℓ проходов сортировки подсчётом. Сперва — по наименее значимому разряду. Затем, на втором проходе, по следующему — а поскольку сортировка подсчётом устойчива, элементы с совпадающим вторым разрядом останутся отсортированными по первому. И так далее, по порядку, вплоть до самого старшего разряда. Каждый проход — $O(n + k)$, всего — $O(\ell(n + k))$.



Рис. 2: Чарльз Доджсон (Льюис Кэрролл) (1832–1898).

Пример 3. Исходный массив: 225, 145, 244, 125, 143, 123, 243.
 После сортировки по единицам: 143, 123, 243, 244, 225, 145, 125.
 После сортировки по десяткам: 123, 225, 125, 143, 243, 244, 145.
 После сортировки по сотням: 123, 125, 143, 145, 225, 243, 244.

4 Нахождение i -го по величине элемента массива

Пусть в массиве $a_1, \dots, a_n$ надо найти наибольший элемент. Это можно сделать за $n - 1$ сравнений. Наименьший — тоже за $n - 1$. Аналогия: спортивный турнир, в котором участники соревнуются попарно, реализуя сравнение двух элементов. В соревнованиях бывает нужно найти не только наибольший элемент, но сразу несколько первых по величине элементов, за минимальное количество партий. К математическому исследованию этой задачи, в частности, приложил руку знаменитый писатель Льюис Кэрролл (Доджсон [1883]).

Нахождение первого и второго по величине элементов: можно сделать за $n + \lceil \log_2 n \rceil - 2$ сравнений (Шрейер), и такое количество сравнений необходимо (Кислицин).

Упражнение 1. Придумать алгоритм, находящий наименьший и наибольший элементы массива за $\frac{3}{2}n$ сравнений.

Как найти медиану — то есть, элемент находящийся ровно посередине в отсортированном массиве? Более общая задача: найти i -й по порядку элемент в отсортированном массиве (его ещё называют i -й «порядковой статистикой», order statistic). Можно, конечно, просто отсортировать массив за время $O(n \log n)$, а потом взять i -й элемент получившегося отсортированного массива. Но можно быстрее!

Алгоритм нахождения i -го по величине элемента массива $a_1, \dots, a_n$ (алгоритм 3) сперва делит массив на $\lceil \frac{n}{5} \rceil$ пятёрок элементов, из которых последняя, возможно, неполная.

$$\underbrace{a_1, a_2, a_3, a_4, a_5}_{1\text{-я пятёрка}}, \underbrace{a_6, a_7, a_8, a_9, a_{10}}_{2\text{-я пятёрка}}, \dots, \underbrace{\dots, a_n}_{\lceil \frac{n}{5} \rceil\text{-я пятёрка}}$$

В каждой пятёрке находится медиана. Берётся массив из $\lceil \frac{n}{5} \rceil$ медиан, и в нём тем же методом рекурсивно определяется медиана y . Это первый рекурсивный вызов. Далее ко всему массиву применяется процедура partition(), как в алгоритме «быстрой сортировки»,



Рис. 3: Мануэль Блум (род. 1938), Роберт Флойд (1936–2001), Вон Пратт (род. 1944), Рональд Райвест (род. 1947), Роберт Тарьян (род. 1948).

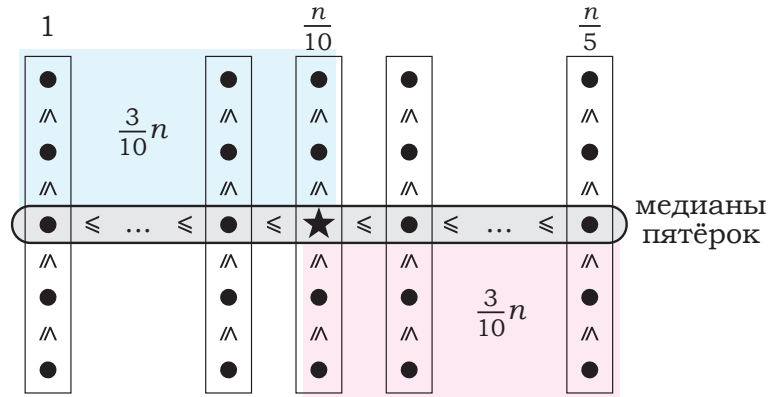


Рис. 4: Пятёрки, отсортированные по медиане; выделены $\frac{3}{10}n$ элементов, заведомо не превосходящих медиану медиан, и $\frac{3}{10}n$ элементов, заведомо не меньших, чем медиана медиан.

используя медиану медиан в качестве опорного элемента. Если медиана медиан оказалась i -м элементом массива, то она и возвращается. Иначе берётся та часть массива, в которую попадает i -я позиция, и к ней рекурсивно применяется тот же алгоритм. Это второй рекурсивный вызов.

Оценка времени работы. Надо оценить функцию $T(n)$ — максимальное число сравнений при работе с n -элементным массивом. Медиана медиан находится за $T(\lceil \frac{n}{5} \rceil)$ действий. Пусть y — медиана медиан. Тогда в массиве есть хотя бы $\frac{3}{10}n - O(1)$ элементов, не превосходящих y : действительно, медианы половины пятёрок не превосходят y , и, стало быть, в этих $\lfloor \frac{n}{10} \rfloor$ пятёрок есть хотя бы $3 \cdot \lfloor \frac{n}{10} \rfloor$ элементов, не меньших, чем y . Аналогично, есть хотя бы $\frac{3}{10}n - O(1)$ элементов, не меньших, чем y . Потому процедура, вызываемая рекурсивно, применяется к массиву размера не более чем $\frac{7}{10}n + O(1)$.

Отсюда $T(n) = T(\frac{n}{5}) + T(\frac{7n}{10}) + O(n)$. Здесь самое главное то, что $\frac{n}{5} + \frac{7n}{10} < n$ — стало быть, чем глубже уровень рекурсии, тем меньше на нём работы. Поэтому функцию $T(n)$ получится оценить сверху линейной функцией. Это делается так: пусть $T(n) < T(\frac{n}{5}) + T(\frac{7n}{10}) + C \cdot n$ для некоторой константы $C > 0$; утверждается, что $T(n) < 20 \cdot C \cdot n$. Действительно,

$$T(n) < 4Cn + 14Cn + Cn = 19Cn$$

Вопрос для самостоятельных размышлений: Можно ли вместо пятёрок использовать тройки или семёрки?

Алгоритм 3 Нахождение i -го по величине элемента массива (Блум, Флойд, Пратт, Райвест, Тарьян [1973])

Процедура $\text{select}(a_1, \dots, a_n; i)$

```

1: if  $n = 1$  then
2:   return  $a_1$ 
3: Пусть  $k = \lceil \frac{n}{5} \rceil$ 
4: Отсортировать каждую пятёрку  $a_{5j+1}, \dots, a_{5j+5}$ 
5: Пусть  $b_1, \dots, b_k$  — массив, где  $b_j$  — медиана  $j$ -й пятёрки.
6:  $y = \text{select}(b_1, \dots, b_k; \lfloor \frac{k}{2} \rfloor)$  /* медиана медиан */
7:  $j = \text{partition}(a_1, \dots, a_n; y)$ 
8: if  $j = i$  then
9:   return  $i$ 
10: else if  $j > i$  then
11:   return  $\text{select}(a_1, \dots, a_{j-1}; i)$ 
12: else
13:   return  $j + 1 + \text{select}(a_{j+1}, \dots, a_n; i - j - 1)$ 

```

Процедура $\text{partition}(x[0..m-1], y)$

```

1: Пусть  $p$  — наименьший номер элемента  $a_p$ , равного  $y$ 
2:  $x_p \leftrightarrow x_{m-1}$  /* опорный элемент сперва помещается в конец */
3:  $i = 0$ 
4:  $j = m - 1$ 
5: while  $i < j$  do
6:   if  $x_i < y$  then
7:      $i = i + 1$ 
8:   else if  $x_j \geq y$  then
9:      $j = j - 1$ 
10:  else
11:     $x_i \leftrightarrow x_j$ 
12:  $x_{i-1} \leftrightarrow x_{m-1}$  /* переместить опорный элемент в найденную середину */
13: return  $i$  /* опорный элемент теперь находится здесь */

```

5 Метод динамического программирования

5.1 Задача о разделении стержня

Задача о разделении стержня. Имеется стержень длины n . Продав стержень всякой длины i , можно выручить p_i денежных единиц. Как выгоднее всего распилить имеющийся стержень?

На каждом шаге — *принятие решения*, какого размера кусок отпилить. Затем — решение *подзадачи*, за сколько можно продать то, что осталось.

Оптимальный результат можно получить, решая подзадачи рекурсией — но тогда получится слишком много работы.

Алгоритм 4 Наивный алгоритм для задачи о разделении стержня

Процедура разделить(n)

```
1: if  $n = 0$  then
2:   return 0
3: else
4:   return  $\max_{1 \leq i \leq n} (p_i + \text{разделить}(n - i))$ 
```

Другой подход — «жадный алгоритм»: на каждом шаге принимать то решение, которое в данный момент сулит наибольшую выгоду. Здесь: отпилить самый дорогой кусок и продать. Затем так же поступить с тем, что осталось. Работает быстро, но не всегда даёт оптимальный результат.

Пример 4. Пусть $n = 6$, $p_1 = 0$, $p_2 = 2$, $p_3 = 7$, $p_4 = 10$. Жадный алгоритм заметит, что куски размера 4 ценятся дороже всего (по 2.5 за единицу длины) и продаст один такой кусок. Оставшийся после этого кусок размера 2 придётся продавать за 2, и в итоге получится выручить 12. Однако выгоднее разрезать на 3 и 3, продав в совокупности за 14.

Получить точный ответ за небольшое время можно *методом динамического программирования*. Термин «динамическое программирование» (Беллман, 1950-е) относится к принятию решений — составлению программы действий. Суть этого метода такова. Пусть на каждом шаге надо сделать выбор (принять решение). Известно, что *какой-то* выбор приводит к оптимальному результату. Этому выбору соответствует некий набор подзадач. Тогда сперва находятся ответы для всех подзадач данной задачи, возникающих при различном выборе, после чего, имея все эти ответы перед глазами, можно будет в каждом случае сделать наилучший выбор.

Алгоритм 5 использует метод динамического программирования для решения задачи о разделении стержня за квадратичное время от длины стержня n . Алгоритм последовательно вычисляет максимальную выручку от продажи стержней длины $1, \dots, n$, размещая результат для длины j в переменной r_j . При этом в момент вычисления r_j все значения $r_1, \dots, r_{j-1}$ уже вычислены и используются для нахождения наилучшего разреза стержня длины j .

Стоит заметить, что зависимость времени работы алгоритма от *размера* входных данных — то есть, от количества битов входных данных — экспоненциальная. Такие алгоритмы называют *псевдополиномиальными*.

Алгоритм 5 Динамическое программирование для задачи о разделении стержня

Дано: длина стержня n , цены на отрезки $p_1, \dots, p_n$.

Для всех $j \in \{0, \dots, n\}$, вычислить r_j — наибольшую сумму, которую можно выручить, разрезав и продав стержень длины j .

```
1:  $r_0 = 0$                                      /* стержень длины 0 не стоит нисколько */
2: for  $j = 1$  to  $n$  do
3:    $r_j = 0$                                      /* пока не найдено никаких способов продать стержень */
4:   for  $i = 1$  to  $j$  do                           /* для всякой длины отрезаемого куска */
5:      $r = p_i + r_{j-i}$                            /* сложить его цену с выручкой за остаток */
6:     if  $r > r_j$  then                             /* если так можно выручить больше известного */
7:        $r_j = r$                                    /* ... то оценка цены стержня длины  $j$  улучшается */
8: return  $r_n$                                      /* столько можно выручить за весь стержень */
```

Список литературы

- [1973] M. Blum, R. W. Floyd, V. Pratt, R. L. Rivest, R. E. Tarjan, “Time bounds for selection”, *Journal of Computer and System Sciences*, 7:4 (1973), 448–461.
- [1883] C. L. Dodgson, “Lawn tennis tournaments: The true method of assigning prizes with a proof of the fallacy of the present method”, *St. James’s Gazette*, 1 августа 1883 г., стр. 5–6.
- [1961] C. A. R. Hoare, “Algorithm 64: Quicksort”, *Communications of the ACM*, 4:7 (1961), 321.
- [1981] C. A. R. Hoare, “The emperor’s old clothes”, *Communications of the ACM*, 24:2 (1981), 75–83.
- [1964] J. W. J. Williams, “Algorithm 232: Heapsort”, *Communications of the ACM*, 7:6 (1964), 347–348.